

CODEFEST AD ASTRA 2026

Documento tecnico — Etapa 1: Base de Conocimiento Vectorial

1. El corpus y su preprocesamiento

El corpus entregado por ADL son **1826 documentos** (3 GB) de 21 observatorios repartidos en los tres fenomenos: 760 PDF, 964 JSON, 26 CSV, 73 PBF, 8 imagenes, 6 XLSX y 1 TXT. La heterogeneidad es el primer problema de ingenieria del reto: un mismo indice tiene que absorber informes de 400 paginas de SIPRI o del AI Index, articulos web serializados en JSON, datasets bibliograficos de decenas de MB y teselas de mapas vectoriales. De los 1826 documentos, **1818 aportan texto**; los 8 restantes son 5 imagenes sin texto legible, un JSON vacio y dos archivos con extension *.pdf* que en realidad son paginas HTML de error de la descarga.

Cada formato de origen tiene un extractor dedicado que produce un objeto comun (*RawDocument*) con el texto crudo y metadata auxiliar: **PDF** (pypdfium2, preservando el orden de lectura por pagina y con deteccion heuristica de boilerplate — lineas identicas repetidas en $\geq 30\%$ de las paginas de un documento de 3 o mas paginas se descartan como cabecera/pie de pagina), **HTML** (trafilatura, que separa el contenido principal del menu de navegacion, anuncios y pie de pagina, y preserva encabezados como senales Markdown), **JSON** (mapeo explicito de campos conocidos como *title/body_paragraphs/body_text*, con fallback generico para esquemas no anticipados), **CSV/XLSX** (cada fila se convierte en pares *columna: valor* y se trata como unidad atomica de fragmentacion) e **imagenes** (OCR opcional via pytesseract, con degradacion controlada si el binario de tesseract no esta disponible). Tres decisiones merecen justificacion explicita porque las impuso el corpus real:

- **pypdfium2 en vez de pdfplumber** para los PDF. Con 760 PDF y del orden de 30.000 paginas, el costo de extraccion deja de ser irrelevante: medido sobre este corpus, pypdfium2 lee unas 3.000 paginas en 4,5 s, mientras que pdfplumber es dos ordenes de magnitud mas lento y convierte la fase offline de minutos en horas. Para texto plano de informes la calidad extraida es equivalente.
- **OCR selectivo.** Cerca del 5% de los PDF — casi todos los informes de riesgo de la Defensoria del Pueblo — son escaneos sin capa de texto. Cuando un documento rinde menos de 200 caracteres por pagina se asume escaneo y se rasteriza para pasarlo por tesseract, con un tope de 60 paginas por documento: sin ese tope un puñado de escaneos largos domina el tiempo total de la corrida.
- **Los .pbf no son OpenStreetMap.** Las 73 teselas de Amazon Underworld son *Mapbox Vector Tiles*, no el formato Protocol Buffer de OSM, asi que las librerias habituales (pyosmium, pyrosm) no las leen. Se decodifican con *mapbox-vector-tile*, se recorren las capas leyendo los atributos de cada elemento como pares *atributo: valor*, y se deduplican los elementos repetidos entre niveles de zoom, tal como indica la sec. 2.1.

Los CSV/XLSX llevan un tope de 500 filas por archivo. Los datasets del AI Index son volcados bibliograficos de PubMed de hasta 35 MB, ajenos a las consultas del reto: sin tope aportarian mas de 100.000 fragmentos de ruido que dominarian el espacio vectorial. Con tope, el documento sigue indexado y puede recuperarse a nivel documento, pero no ahoga al resto del corpus. La limpieza posterior (normalizacion Unicode NFC, remocion de caracteres de control, deteccion de idioma) es deliberadamente conservadora sobre el contenido — sin lowercasing ni stemming — porque la evaluacion compara el campo *text* de forma textual contra el ground truth.

1.1 Identificacion de documentos (doc_id)

El emparejamiento a nivel documento se hace con el **DOC_ID** que **suministra ADL** en el inventario del corpus (formato *F1-AIINDEX-001*), no con un identificador propio. El manifest se indexa por **ruta relativa** y no por nombre de archivo: 59 nombres del inventario aparecen en dos carpetas distintas con DOC_ID distintos — los informes de CSET bajo *pdfs/Reports* y *pdfs/Translation*, y las teselas que se repiten por nivel de zoom — de modo que indexar por nombre le habria asignado el identificador equivocado a 127 documentos y anulado su aporte al F1@3. Once archivos presentes en el corpus no figuran en el inventario de ADL (catalogos y registros del proceso de descarga); al no tener DOC_ID no pueden aparecer en el ground truth, y se excluyen del indice para que no ocupen uno de los tres cupos de documento por consulta.

2. Estrategia de chunking y justificacion

Se implemento una estrategia **hibrida estructural + oracional con solapamiento**, en tres niveles:

- **Estructural:** el texto se separa primero por encabezados Markdown (que HTML/JSON/MD ya traen marcados por el extractor); si el documento no tiene encabezados (caso tipico de PDF), se trata como una unica seccion.
- **Segmentacion de oraciones multilingue:** dentro de cada seccion, se usa *pysbd* (reglas linguisticas dedicadas) para espanol e ingles, y el parser entrenado de spaCy (*pt_core_news_sm*) para portugues, que *pysbd* no cubre nativamente.
- **Empaquetado voraz por presupuesto de tokens** (~280 tokens, margen bajo el limite de 512 del encoder), contando tokens con el tokenizer real del encoder de indexacion — no una aproximacion por palabras — con **solapamiento de 1 oracion** entre chunks consecutivos para no perder ideas que caen en la frontera.

La unidad minima que se mueve entre chunks es siempre una oracion completa: el empaquetador nunca corta un caracter a mitad de oracion, cumpliendo el requisito de completitud linguistica (sec. 3.3). Esta garantia se verifica automaticamente en *tests/test_chunking.py*. La unica excepcion estructural es CSV/XLSX, donde cada fila (no una oracion en lenguaje natural) se indexa como chunk atomico, tal como permite explicitamente la especificacion.

3. Encoder(s) seleccionado(s) y criterios de eleccion

Encoder primario: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2** (arquitectura encoder tipo BERT, licencia Apache 2.0).

Criterio	Justificacion
Soporte multilingue	Cubre nativamente espanol, ingles y portugues (50+ idiomas), necesario porque el corpus y las consultas mezclan los tres.
Dimensionalidad	384 dimensiones: balance razonable entre costo de almacenamiento/busqueda y expresividad para el volumen esperado del reto.
Longitud maxima de entrada	512 tokens; el presupuesto de chunking (~280 tokens) deja margen suficiente.
Licencia	Apache 2.0, uso libre sin restricciones para el contexto del reto.
Eficiencia computacional	Modelo liviano (backbone MiniLM), adecuado para los recursos de computo limitados del equipo.

Riesgo de reproducibilidad	Ya validado en el material de apoyo entregado por el equipo organizador (01_embeddings.ipynb, 02_rag.ipynb), reduciendo el riesgo de incompatibilidades de ultimo momento.
----------------------------	--

3.1 Encoder secundario y fusion de rankings (sec. 4.4)

El pipeline soporta construir la base con mas de un encoder. El secundario es **intfloat/multilingual-e5-base** (encoder tipo BERT, licencia MIT, 768 dimensiones, multilingue nativo, limite 512 tokens). El criterio de seleccion no fue elegir un modelo "mejor" en abstracto, sino uno con **errores decorrelacionados** respecto al primario: E5 esta entrenado para recuperacion densa mientras que el primario esta afinado para similitud de parafrasis. Fusionar dos rankings solo aporta cuando los modelos se equivocan en casos distintos; dos encoders de la misma familia habrian aportado poco.

La familia E5 exige prefijos distintos para consulta (*query:*) y pasaje (*passage:*); omitirlos degrada la calidad silenciosamente. Por eso la interfaz *Encoder* distingue codificacion asimetrica (*encode_query* / *encode_passages*), y los encoders que no los requieren heredan el comportamiento por defecto.

Las listas se combinan con **Reciprocal Rank Fusion** (RRF, ecuacion 7, con $k_0=60$), elegido sobre CombSUM/CombMNZ porque opera sobre posiciones y no sobre puntuaciones absolutas: es robusto a que dos encoders produzcan similitudes en escalas distintas. El grafo de conocimiento se incorpora como una lista adicional a fusionar, tal como sugiere la sec. 8.5.

Invariante de correccion: la fusion empareja fragmentos por *chunk_id*, lo que solo es valido si todos los indices comparten los mismos fragmentos. Como el presupuesto de chunking depende del tokenizero, fragmentar por separado con cada encoder produciria *chunk_id* colisionantes apuntando a textos distintos. Por eso el chunking se ejecuta una unica vez y sus fragmentos se reutilizan para todos los encoders; *generador.py* ademas verifica la coincidencia al cargar y aborta si detecta indices desincronizados, en lugar de producir resultados silenciosamente incorrectos.

Como el *ground truth* oficial no es publico, la decision entre uno y dos encoders no se toma por preferencia sino contra el ground truth reducido descrito en la seccion 7, complementado con *scripts/compare_encoders.py*, que cuantifica el solapamiento entre ambos rankings: un solapamiento alto indica que fusionar no aportaria y que conviene entregar un solo indice. Mientras esa comparacion no arroje una mejora medible, la configuracion multi-encoder queda detras de un flag y desactivada por defecto — se prefiere entregar la configuracion simple verificada antes que la compleja no verificada.

No se emplea ningun modelo decoder/generativo en ninguna etapa de indexacion o recuperacion (sec. 4.2 y 8.3): en particular, se descarto explicitamente HyDE (requiere un LLM para generar una respuesta hipotetica) y el reranking con LLM. El uso de un *cross-encoder* (arquitectura encoder, no generativa, demostrado en el material de apoyo) quedo implementado detras de un flag desactivado por defecto, como exploracion documentada, para no depender de el en el flujo evaluado.

4. Indice FAISS

Se utiliza **IndexFlatIP** sobre vectores normalizados a norma unitaria, equivalente a similitud coseno (sec. 5.2 y 8.2). Es la opcion recomendada por la propia especificacion para el volumen de documentos esperado en esta etapa del reto: garantiza resultados exactos (sin la perdida de exactitud de IVFFlat/HNSW) y su costo computacional es totalmente asumible a esta escala. El identificador interno de FAISS se mantiene alineado linea a linea con *metadata.jsonl* (mismo orden de insercion que de escritura), invariante verificado en *tests/test_faiss_alignment.py*. Si el corpus real de ADL resultara mucho mayor al

esperado, *src/config.py* centraliza el punto donde migrar a un índice aproximado sin tocar el resto del pipeline.

La decisión se verificó empíricamente antes de fijarla. Con vectores de 384 dimensiones y $k=120$ sobre CPU, una búsqueda exacta tarda 1,9 ms (mediana) con 20.000 vectores, 5,7 ms con 50.000 (9,8 ms en el percentil 99) y 13,2 ms con 100.000. El corpus real produce **128.526 fragmentos**, es decir del orden de 15 ms por consulta: irrelevante frente a los ~27 ms que cuesta vectorizar la consulta y los ~19 s de carga del encoder. Migrar a IVFFlat o HNSW habría cambiado búsqueda exacta por búsqueda aproximada — perdiendo recall, que es justamente lo que mide la evaluación — a cambio de ahorrar milisegundos que nadie percibe.

5. Grafo de conocimiento (componente bonus)

NER: se usa el componente de reconocimiento de entidades ya entrenado en los modelos spaCy *es/en/pt_core_news_sm* (licencia MIT, arquitectura no generativa), filtrando tipos de entidad puramente numéricos/temporales (fechas, cantidades, porcentajes) para quedarse con personas, organizaciones, lugares y otras entidades nombradas relevantes.

Extracción de relaciones: heurística basada en dependencias sintácticas (permitida explícitamente por la especificación como alternativa a un modelo entrenado de RE): para cada oración con al menos dos entidades, se toma el verbo/auxiliar entre un par de entidades consecutivas como etiqueta de relación (o el verbo raíz de la oración si no hay uno intermedio), con una relación genérica de co-ocurrencia como último recurso.

Construcción e integración: las tripletas se acumulan en un *networkx.MultiDiGraph*, cada arista con *doc_id*, *chunk_id* y la relación como evidencia trazable, exportado a GraphML. En recuperación, las mismas entidades se detectan en la consulta, se buscan sus vecinos de primer orden en el grafo, y los chunks vinculados se fusionan con el ranking vectorial mediante Reciprocal Rank Fusion (el grafo se trata como un índice adicional, sec. 8.5), detrás de un flag *--use-graph* opcional en *generador.py*.

6. Módulo de recuperación

- Mismo encoder para indexación y consulta; vector de consulta normalizado igual que el índice.
- Sobre-recuperación de candidatos antes de aplicar post-filtros de metadata (fenómeno, formato, idioma) o de umbral de similitud coseno (sec. 8.7).
- Agregación a nivel documento por **suma** de las puntuaciones de sus fragmentos (configurable a máximo o media) sobre un pool de 60 candidatos, más amplio que los 10 fragmentos mostrados, para que la relevancia de un documento no dependa solo de si su mejor chunk aparece en el top-10 (sec. 8.6). La elección de la suma sobre el máximo esta medida, no supuesta: ver sección 7.
- Reciprocal Rank Fusion ($k_0=60$) como mecanismo único de combinación, reusado tanto para múltiples encoders como para vector+grafo (sec. 8.4).
- Límite de 250 palabras por fragmento aplicado dividiendo en límites oracionales completos cuando un chunk lo excede; los sub-fragmentos comparten *chunk_id* y ocupan su propio *rank* (sec. 9.2.1). El corpus real obliga a agregar un último recurso: en texto proveniente de OCR la puntuación puede desaparecer por completo y el segmentador devuelve una única "oración" de más de 250 palabras, que respetar el límite oracional dejaría pasar entera. En ese caso — y solo en ese — el corte se hace por palabras.

Ninguna etapa de recuperacion usa un modelo generativo: solo vectores, puntuaciones de similitud coseno y aritmetica sobre metadata.

7. Como se tomaron las decisiones: medicion interna

El ground truth oficial no es publico durante el reto, de modo que ninguna decision de diseno puede validarse contra la metrica real. Para no elegir a ojo se anotaron a mano **10 de las 50 consultas oficiales**, repartidas en los tres fenomenos, marcando los documentos relevantes de cada una (*dev/eval/*). Sobre esa base, *scripts/barrido_retrieval.py* carga el indice una vez y evalua en memoria todas las combinaciones de tamano de pool y estrategia de agregacion.

El resultado que cambio la configuracion entregada: con pool de 60 candidatos, agregar por **suma** en lugar de por **maximo** sube el F1@3 de **0,200 a 0,300**. La lectura es que un documento verdaderamente relevante para una consulta contiene varios pasajes relevantes, mientras que el maximo premia al documento que tuvo un unico fragmento afortunado. La suma resulto igual o mejor que el maximo en practicamente toda la grilla explorada, lo que sostiene la decision mas alla del valor puntual.

Lo que esta medicion no permite concluir. Con 10 consultas, que una sola cambie de resultado mueve el promedio en 0,1: el tamano del pool oscilo entre 0,20 y 0,30 sin tendencia monotona, de modo que el valor 60 esta dentro de una meseta y no es un optimo demostrado. Se documenta explicitamente porque la alternativa — presentar cada numero de un barrido como un hallazgo — llevaria a sobreajustar el sistema a diez consultas y degradar las cuarenta restantes. Por la misma razon se descarto una hipotesis intermedia sobre recuperacion entre idiomas distintos: la muestra no permitia distinguirla del ruido.

La entrega se verifica de punta a punta antes de empaquetarse con *scripts/validar_entrega.py*, que comprueba la estructura de carpetas, las 50 lineas, los 3 documentos y 10 fragmentos por consulta, el limite de 250 palabras, la alineacion entre el indice FAISS y *metadata.jsonl*, y que todo *doc_id* reportado pertenezca al inventario de ADL. La suite de pruebas (51 casos) cubre los invariantes criticos, incluida la ejecucion de *generador.py* como subprocesso con *PYTHONPATH* vacio para garantizar que es autocontenido y que los resultados son reproducibles, requisito del punto 4 de la sec. 1.4.

8. Limitaciones conocidas y decisiones documentadas

- **El ground truth reducido es la limitacion dominante del proyecto.** Los valores de F1@3 de la seccion 7 sirven para ordenar configuraciones entre si, no para estimar la nota: la anotacion es parcial, asi que un documento relevante no anotado cuenta como error y el valor real sera mayor. Diez consultas tampoco alcanzan para calibrar mas de un parametro sin sobreajustar; ampliar la anotacion es la mejora de mayor impacto pendiente, por encima de cualquier cambio adicional al recuperador.
- **Documentos sin texto recuperable:** 8 de los 1826. Cinco son imagenes (una de ellas en formato AVIF, que Pillow no lee sin plugin adicional), una es un JSON de 0 bytes y dos son paginas HTML de error guardadas con extension .pdf. Estos ultimos se detectan por contenido y se procesan como HTML, pero su texto es una pagina de error y no aporta nada. Estos documentos quedan en el corpus sin posibilidad de ser recuperados.
- **Campo fuente:** se reporta siempre el nombre de archivo estandarizado del inventario de ADL, nunca la URL de origen (que se conserva aparte, en un campo *url* adicional). La sec. 10.2.1 sugeria *fuentes* como clave de emparejamiento y la aclaracion posterior de ADL indico *doc_id*; reportar el nombre de archivo hace que el sistema funcione bajo cualquiera de las dos lecturas.

- **Fusion de chunks cortos adyacentes** (sec. 9.2.1, mencionada como posible, no obligatoria) no esta implementada: solo se implemento la division obligatoria de chunks que exceden 250 palabras.
- **Boilerplate de PDF:** la heuristica de lineas repetidas ($\geq 30\%$ de paginas) requiere documentos de 3 o mas paginas; PDFs mas cortos no reciben este filtro.